

Expression is a mathematic statement that consist of operand and operator. Normally we use Infix Expression.

$$2 + 3 * 5 = 17 \quad (\text{ลำดับความสำคัญมาตรฐาน PEMDAS: คูณก่อนบวก } 2 + 15 = 17)$$

$$2 + 3 * 5 = 25 \quad (\text{กรณีคำนวณจากซ้ายไปขวาโดยไม่คำนึงถึง Precedence: } 5 * 5 = 25 \text{ [ข้อผิดพลาดที่พบบ่อย]})$$

Precedence rules

Python's arithmetic operator precedence follows the standard mathematical order of operations, often remembered by the acronym PEMDAS:

- **P**arentheses `()`: Expressions within parentheses are evaluated first, overriding other precedence rules.
- **E**xponents `**`: Exponentiation operations are performed next.
- **M**ultiplication `*`, **D**ivision `/`, Floor Division `//`, Modulo `%`: These operators have the same precedence and are evaluated from left to right.
- **A**ddition `+`, **S**ubtraction `-`: These operators have the lowest precedence among arithmetic operators and are evaluated from left to right.

Example:

Python

```
result = 3 + 4 * 5
```

In this example, the multiplication `4 * 5` is performed first due to higher precedence, resulting in 20. Then, `3 + 20` is calculated, yielding 23.

Using Parentheses to Override Precedence:

Python

```
result = (3 + 4) * 5
```

Here, the addition `3 + 4` is performed first because it is enclosed in parentheses, resulting in 7. Then, `7 * 5` is calculated, yielding 35.

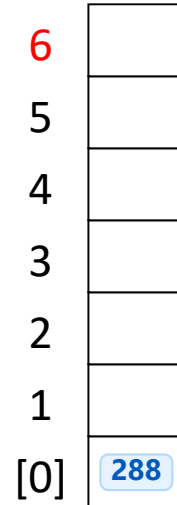
Postfix expression

How to evaluate a postfix expression (read postfix.pdf in google classroom)

6 5 2 3 + 8 * + 3 + * = 288

[Trace ตารางคำนวณ Postfix Evaluation ทีละขั้นตอน]:

* อ่าน 6, 5, 2, 3 -> Push เข้า Stack: [6, 5, 2, 3]
 * อ่าน '+' -> Pop 3, 2 -> 2 + 3 = 5 -> Push 5 (Stack: [6, 5, 5])
 * อ่าน 8 -> Push 8 (Stack: [6, 5, 5, 8])
 * อ่าน '*' -> Pop 8, 5 -> 5 * 8 = 40 -> Push 40 (Stack: [6, 5, 40])
 * อ่าน '+' -> Pop 40, 5 -> 5 + 40 = 45 -> Push 45 (Stack: [6, 45])
 * อ่าน 3 -> Push 3 (Stack: [6, 45, 3])
 * อ่าน '+' -> Pop 3, 45 -> 45 + 3 = 48 -> Push 48 (Stack: [6, 48])
 * อ่าน '*' -> Pop 48, 6 -> 6 * 48 = 288 -> Push 288 (Stack: [288])
 -> ผลลัพธ์สุดท้าย: 288 (Stack เหลือตัวเดียวคือคำตอบที่สมบูรณ์)



Algorithm:

- use a stack
- a number is seen → push onto stack
- an operator is seen → pop two values from stack
 apply operation
 push the result onto stack

[สรุปอัลกอริทึมและฟังก์ชันประเมินค่า Postfix ในภาษา Python]:

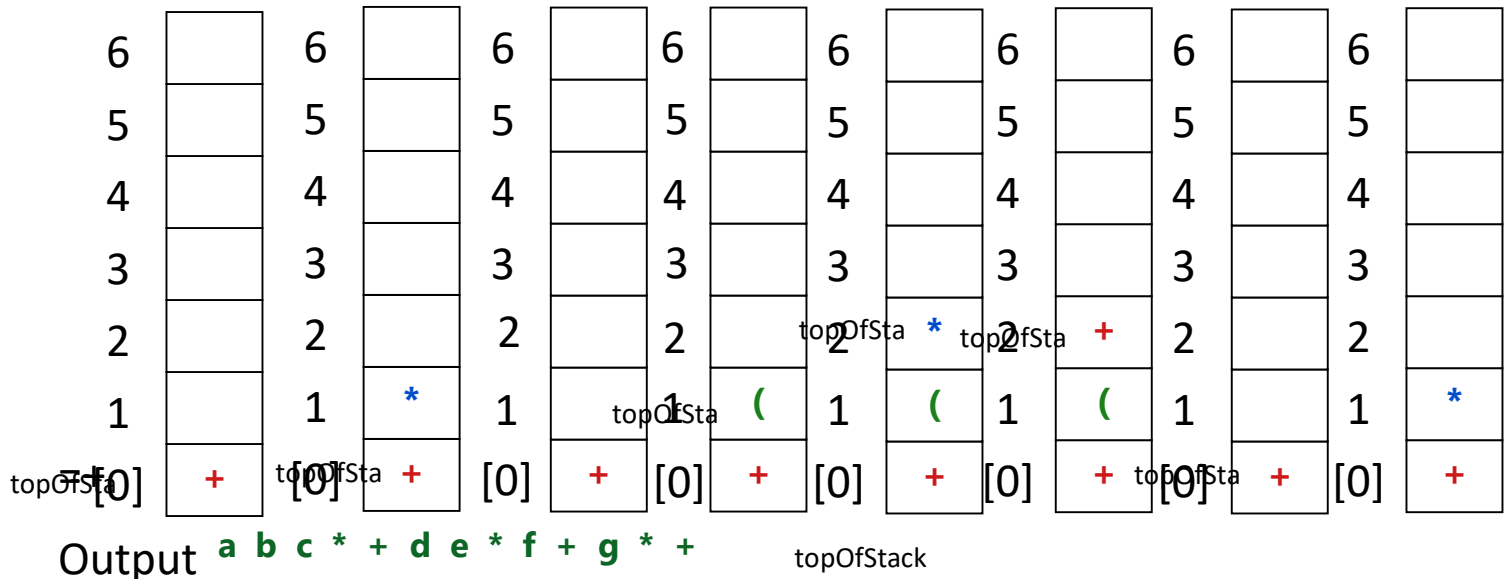
* หลักการสำคัญ: ใช้ Stack เก็บ Operand เมื่อเจอ Operator ให้ Pop 2 ค่ามาคำนวณแล้ว Push ผลลัพธ์กลับ
 * ความซับซ้อน: Time O(N) เพราะประมวลผลทีละรอบเดียว และ Space O(N)
 * โค้ดภาษา Python สำหรับประเมินค่า Postfix Expression:

```
def evaluate_postfix(tokens):
    stack = []
    for token in tokens.split():
        if token.isdigit(): stack.append(int(token))
        else:
            b, a = stack.pop(), stack.pop()
            if token == '+': stack.append(a + b)
            elif token == '*': stack.append(a * b)
    return stack.pop() # evaluate_postfix('6 5 2 3 + 8 * + 3 + *') == 288
```

Infix to Postfix Conversion (read postfix.pdf in google classroom)

$a + b * c + (d * e + f) * g$

/



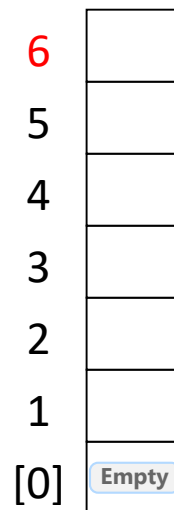
[ตาราง Shunting-Yard Trace ที่ละโทเคน]:

* กฎหลักการแปลง Infix -> Postfix (Shunting-Yard Algorithm):

1. เจอ Operand (a..g) -> ส่งออกไปที่ Output ทันที
2. เจอ '(' -> Push ลง Stack ทันที (ลำดับความสำคัญสูงสุดเมื่ออยู่นอก Stack)
3. เจอ ')' -> Pop ตัวดำเนินการออกจาก Stack ส่งไป Output จนกว่าจะเจอ '('
4. เจอ Operator (+, *) -> Pop ตัวบนสุดที่มี Precedence สูงกว่าหรือเท่ากันออก ก่อนที่จะ Push ตัวดำเนินการใหม่ลง Stack

* การ Trace ที่ละขั้นตอนสำคัญ:

- a -> Output: a | + -> Stack: [+]
- b -> Output: a b | * -> Stack: [+ , *]
- c -> Output: a b c | + -> Pop '*' และ Pop '+' -> Output: a b c * + | Push '+'
- (-> Stack: [+ , (| d -> Output: ... d | * -> Stack: [+ , (, *]
- e -> Output: ... d e | + -> Pop '*' -> Output: ... d e * | Push '+'
- f -> Output: ... d e * f |) -> Pop '+' -> Output: ... d e * f +
- * -> Stack: [+ , *] | g -> Output: ... d e * f + g
- จบอินพุต -> Pop '*' และ '+' ออกหมด -> ได้: a b c * + d e * f + g * +



For Example 4: Convert infix expression to postfix expression and evaluate that postfix expression.

$(a+(b*c))+(((d*e)+f)*g)$

[ส่วนที่ 1: ขั้นตอนการแปลง Infix -> Postfix โดยใช้ Stack (Shunting-Yard)]:

* นิพจน์ Infix ต้นฉบับ: $(a + (b * c)) + (((d * e) + f) * g)$

* กฎการแปลง (Infix to Postfix Algorithm):

1. Operand (a, b, c, d, e, f, g) ส่งออกไปที่ Output ทันทีโดยไม่ต้องเข้า Stack
2. วงเล็บเปิด '(' push ลง stack เสมอ
3. เมื่อเจอวงเล็บปิด ')' ให้ pop ตัวดำเนินการส่งออกไป Output จนกว่าจะเจอ '(' แล้วทิ้งวงเล็บทั้งคู่
4. ตัวดำเนินการ (+, *) เปรียบเทียบลำดับความสำคัญ (Precedence) ก่อน push

* การแปลงแยกตามก้อนวงเล็บหลัก:

- ก้อนซ้าย: $(a + (b * c)) \rightarrow a b c * +$
- ก้อนขวา: $(((d * e) + f) * g) \rightarrow d e * f + g *$
- เชื่อมด้วย '+' ตรงกลางที่ถูก pop ออกมาลำดับสุดท้าย

-> ผลลัพธ์ Postfix Expression ที่สมบูรณ์: $a b c * + d e * f + g * +$

[ส่วนที่ 2: การประเมินค่า (Evaluation) ด้วย Stack และตัวอย่างเชิงตัวเลข]:

* อัลกอริทึมการประเมินค่า Postfix Expression (Stack Evaluation):

- อ่านทีละโทเคนจากซ้ายไปขวา: หากเป็นตัวแปร/ตัวเลข ให้ Push ลง Stack
- หากพบตัวดำเนินการ ให้ Pop โอเปอเรนด์ 2 ตัวบนสุดมาคำนวณ แล้ว Push ผลลัพธ์กลับลง Stack

* ลำดับการประเมินค่าแบบนิพจน์ทางพีชคณิต (Algebraic Evaluation Trace):

1. $T1 = b * c \rightarrow T2 = a + T1$ (ก้อนซ้าย = $a + b * c$)
2. $T3 = d * e \rightarrow T4 = T3 + f \rightarrow T5 = T4 * g$ (ก้อนขวา = $(d * e + f) * g$)
3. $Result = T2 + T5 = (a + (b * c)) + (((d * e) + f) * g)$

* ตัวอย่างการแทนค่าตัวเลขจริงในห้องเรียน (Concrete Numerical Trace):

- กำหนดให้: $a = 1, b = 2, c = 3, d = 4, e = 5, f = 6, g = 7$
- คำนวณก้อนซ้าย: $T1 = 2 * 3 = 6 \rightarrow T2 = 1 + 6 = 7$
- คำนวณก้อนขวา: $T3 = 4 * 5 = 20 \rightarrow T4 = 20 + 6 = 26 \rightarrow T5 = 26 * 7 = 182$
- ผลรวมสุดท้าย (Final Evaluation Result): $7 + 182 = 189$

-> ข้อสรุปสำคัญ: รูปแบบ Postfix ไม่จำเป็นต้องใช้วงเล็บ และประมวลผลได้ใน $O(N)$ เสมอ

[สรุปสาระสำคัญและโค้ดตัวอย่างในภาษา Python]:

* โค้ดภาษา Python ย่อสำหรับการแปลงและทดสอบผลลัพธ์:

```
prec = {'+': 1, '-': 1, '*': 2, '/': 2, '(': 0}
# infix_to_postfix('(a+(b*c))+(((d*e)+f)*g)') -> 'a b c * + d e * f + g * +'
# evaluate_postfix('1 2 3 * + 4 5 * 6 + 7 * +') -> 189
```

-> สรุปหลักการ: วงเล็บเปิดมีสิทธิ์เข้า Stack เสมอ แต่วงเล็บปิดเป็นตัวส่งเคลียร์ Stack คืนสู่ Output